

# Class 3: NoSQL Architecture

The Document Model & Distributed Logic

# Part 1: The "Change of Chip"

Leaving Relational Paradigms Behind

# The No-SQL Storage Hierarchy

---

Transitioning to MongoDB requires a fundamental mental shift. It is not merely "SQL without tables," but a completely different approach to organizing data across three main levels:

## 1. Database

The highest level of the hierarchy and the physical container for collections. Each database has its own dedicated set of files on the server's disk, functioning similarly to a schema in SQL.

## 2. Collection

The analog to a relational table. It is a grouping of documents, but unlike SQL tables, collections do not enforce a strict, predefined schema.

## 3. Document

The basic unit of data (equivalent to a row). Consists of key-value pairs stored in BSON. Requires a unique **\_id** (which includes a timestamp for sorting). Capped at a **strict 16 MB limit** to prevent bloated structures.

# Overcoming the Impedance Mismatch

---

## The Relational Problem

The relational model restricts data to simple, flat values within tuples (rows). This creates an **"impedance mismatch"** between the database and the rich, nested data structures used by modern applications. To store a user with multiple addresses, you must shatter a single logical object across multiple tables and JOIN them later.

## The Document Solution

Instead of shattering data, the document model allows developers to store nested records, embedded data, and arrays within a **single entity**. This naturally mimics how modern object-oriented languages (Python, Java) construct objects, drastically reducing development translation effort.

# Dynamic Schemas & Polymorphism

---

Because collections are "**schemaless**," documents residing in the exact same collection do not need to have identical "shapes." Keys and values do not need fixed types or sizes. This allows for true **polymorphism** within the database. Development teams can iterate rapidly without the heavy downtime of relational ALTER TABLE scripts.

Document 1: Address as a simple String

```
{
  "name": "Alice",
  "address": "123 Main St, New York"
}
```

Document 2: Address as a Nested Object

```
{
  "name": "Bob",
  "address": {
    "city": "New York",
    "zipCode": "10001",
    "coordinates": [40.71, -74.00]
  }
}
```

# Collection Organization Best Practices

---

## If schemas are dynamic, why use different collections?

Mixing entirely unrelated data (like user profiles and system logs) is a severe anti-pattern. Always separate collections for these architectural reasons:

-  **Indexing Efficiency:** Indexes in MongoDB are defined and applied at the collection level. Grouping similar documents ensures indexes are built efficiently and queries are highly optimized.
-  **Data Locality:** Grouping related documents together ensures that reading multiple items of the same type requires fewer physical disk seeks, significantly improving read performance.
-  **Application Simplicity:** Mixing document types creates a nightmare for developers, who would have to write complex code to constantly check the "type" or "shape" of a document before processing it.

# Part 2: Under the Hood

JSON vs. BSON and Extended Data Types

# JSON vs. BSON

---

## JSON (JavaScript Object Notation)

Plain text format. Great for human readability and network transit.

**Limitations:** Slow to parse programmatically, takes up more space, and lacks distinction between numbers (everything is a float).

## BSON (Binary JSON)

The actual physical storage format used by MongoDB.

**Advantages:** Highly optimized parsing speed, compact binary representation, and heavily extends standard JSON with specialized data types.



# Extended Data Types: Numbers

---

BSON handles explicit numeric precision, which is strictly essential for enterprise application development.

- # **NumberInt (32-bit integer)**: Used for standard counts, inventory stock, or simple iterative tracking.
- ↑? **NumberLong (64-bit integer)**: Necessary for massive scale. Used for immense serial numbers, global transaction IDs, or Big Data tracking.
- 💾 **Decimal128**: Crucial for financial/monetary calculations. Standard JavaScript floating-point precision fails with currency (e.g.,  $0.1 + 0.2 = 0.30000000000000004$ ). Decimal128 ensures exact precision.

# Extended Data Types: Dates & ObjectId

---

## 📅 Date Objects

BSON has native support for Dates (stored as a 64-bit integer representing milliseconds since Unix epoch).

**Why it matters:** It avoids costly string parsing and allows for direct mathematical queries (e.g., \$gt than a specific timestamp) directly at the database level.

## 🆔 ObjectId

Every document requires an `_id` field. By default, MongoDB assigns a 12-byte BSON ObjectId.



```
64a5b2c9 3f1e 4d3a 9c7b 2f1e4d3a9c7b
|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|
Time  Machine PID  Counter
```

The diagram illustrates the 12-byte structure of a BSON ObjectId. It shows a hexadecimal string '64a5b2c9 3f1e 4d3a 9c7b 2f1e4d3a9c7b' with vertical lines below it indicating the byte boundaries. The structure is divided into three parts: 'Time' (4 bytes), 'Machine PID' (5 bytes), and 'Counter' (3 bytes).

This format guarantees uniqueness across distributed clusters without requiring a central auto-incrementing server.

# Part 3: Distributed Logic

Brewer's Conjecture and Cluster Architectures

# The Physical Problem: Scaling

---

## ↑Scaling Up (Vertical)

Adding more RAM, CPU, or storage to a single server.

**Limits:** Physical hardware limits (you can't buy an infinite RAM chip) and massive financial limits (exponential cost increase).

## ↔Scaling Out (Horizontal)

Distributing the database across a cluster of multiple smaller, cheaper commodity servers.

**The Catch:** Traditional ACID transactions collapse system performance in clustered environments due to cross-network table locking.

# Brewer's Conjecture (CAP Theorem)

---

In a distributed data store, you can only guarantee **TWO** of the following three properties simultaneously:

**C – Consistency:** Every read receives the most recent write or an error. (All nodes see identical data).

**A – Availability:** Every request receives a non-error response, without the guarantee that it contains the most recent write.

**P – Partition Tolerance:** The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.

**The Dilemma: Partition Tolerance is never optional because networks WILL fail. Architects must choose between CP or AP.**

# Architectural Trade-offs in Practice

---

## AP Example (Availability over Consistency)

**Scenario:** Retail Shopping Cart.

It is vital that users can *always* add items to their cart to secure sales, even during a network partition. It relies on **Eventual Consistency** (the system will sync up and resolve discrepancies later).

## CP Example (Consistency over Availability)

**Scenario:** Stock Market Trading System.

It cannot tolerate inconsistent data. If the network fails, it will temporarily sacrifice Availability (triggering a denial of service) rather than let a trade happen on outdated pricing data.

# MongoDB in the CAP Theorem

---

**MongoDB is natively a CP system.**

By design, MongoDB prioritizes **Consistency** and **Partition Tolerance**.

**Split Brain Scenario:** During a severe network partition where nodes lose contact...

**Protection:** The Primary Node will immediately isolate itself and *stop accepting writes*.

This protects absolute data consistency until the cluster successfully elects a new primary node.

# Part 5: Advanced Data Modeling



# The 16MB Dilemma & Relationship Types

---

Every MongoDB document is capped at a strict 16 MB limit. This forces architects to design relationships smartly.



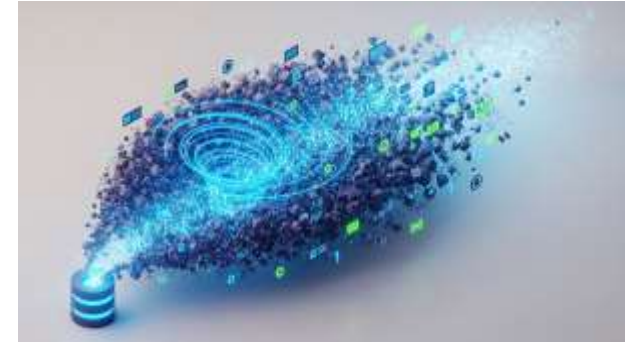
## One-to-Few

A user has 2 or 3 addresses. The data size is small and bounded.



## One-to-Many

A blog post has 500 comments. The data grows, but reasonably.



## One-to-Squillions

A server has millions of event logs. Infinite, unbounded growth.

# Embedding (One-to-Few)

---

When an entity has a small, bounded number of related items, you should Embed them as an array.

## Trade-offs

**Pros:** Incredible read performance via Data Locality. Retrieve the root and all children in one query.

**Cons:** Risk of hitting the 16 MB limit if the array grows infinitely or becomes unbounded.

## JSON REPRESENTATION

```
{
  "name": "Jane",
  "addresses": [
    { "type": "home", "city": "NY" },
    { "type": "work", "city": "LA" }
  ]
}
```

# Referencing (One-to-Many / Squillions)

---

When data is unbounded (e.g., millions of logs), embedding will destroy your database. Use Referencing.

## Architecture & Trade-offs

Store the parent's `ObjectId` in the child documents to maintain normalization.

**Pros:** Unbounded growth support. The parent document stays extremely light and fast.

**Cons:** Requires multiple application-level queries or `$lookup` joins to assemble data.

## NORMALIZED REPRESENTATION

```
// Collection: servers
{ "_id": 1, "name": "Prod-DB" }

// Collection: logs
{ "_id": 991, "server_id": 1, "cpu": 80 }
{ "_id": 992, "server_id": 1, "cpu": 85 }
{ "_id": 993, "server_id": 1, "cpu": 82 }
```

# Part 5: The Aggregation Framework

# What is the Aggregation Framework?

---

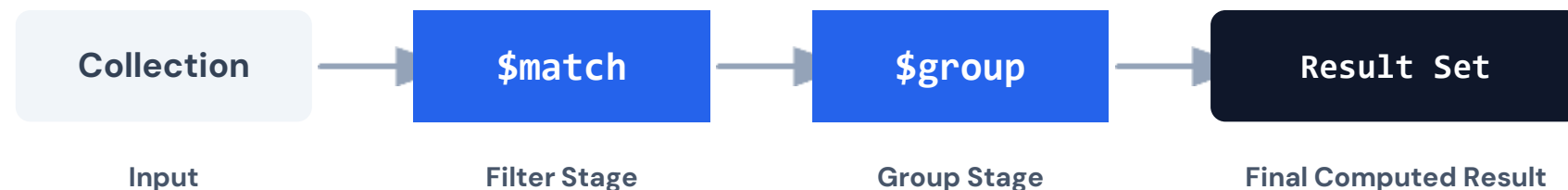
Moving beyond simple `find()` queries for complex data transformations and analytics.

## Advanced Analytics Engine

The Aggregation Framework is modeled on the concept of an **assembly line**.

Documents enter the pipeline, pass through various "stages" that filter, sort, group, and reshape them, and output a final computed result.

### PIPELINE VISUALIZATION



# Core Pipeline Stages

---

Essential operators for transforming and analyzing your data collection.

filter  
\$match

Filters documents to only those that match specified criteria.

Always place first to reduce the data payload in the pipeline.

**SQL: WHERE**

function  
\$group

Groups documents by a specific key.

Performs aggregations like sum, average, or count on group members.

**SQL: GROUP BY**

view  
\$project

Reshapes documents by including or excluding fields.

Can create brand new computed fields for the output.

**SQL: SELECT**

# Shattering the Myth: The \$lookup Stage

---

Unlocking relational capabilities within the MongoDB Aggregation Framework.

da  
ng  
ero  
The Myth  
us

"NoSQL databases cannot do  
JOINS."

ver  
ifie  
\$lookup

**The Reality:** MongoDB introduced the \$lookup stage, which performs an exact **LEFT OUTER JOIN** to an unsharded collection.

It matches the local field to the foreign field and outputs the joined data as a **new embedded array**.

SQL EQUIVALENT: JOIN ... ON ...

# Part 6: Debate & Closing Activity

Applying the Architect's Mindset



# Architectural Debate: AP vs CP

---

If you were the Lead Data Engineer, which architecture (AP or CP) would you mandate for the storage systems of these platforms, and why?



## The Instagram Feed

Millions of posts, likes, and comments per second.



## Hospital Appointment System

Surgical scheduling, doctor availability, and triage.

# Closing Activity: Monolith to Distributed NoSQL

---

*General Scenario Setup: Welcome to your new role as Lead Data Architect for a rapidly scaling global logistics and delivery platform. You are tasked with migrating the company's monolithic relational backend to a distributed No-SQL architecture utilizing MongoDB to handle massive data volumes and clustered environments.*

## Phase 1: CAP Theorem and Distributed Architecture Evaluation

### Subsystem A: Telemetry Ingestion

- Continuous, high-frequency GPS data from millions of vehicles.
- Objective: Determine and technically justify the required distributed architecture (AP or CP).
- Detail behavior during a "split brain" network partition.

### Subsystem B: Financial Ledger

- Processes financial billing and strict chain-of-custody ledger for asset transfers.
- Critically assess how MongoDB's default cluster behavior aligns with requirements.
- Technically justify the necessity of Availability vs. Consistency.

# Closing Activity: Monolith to Distributed NoSQL

*General Scenario Setup: Welcome to your new role as Lead Data Architect for a rapidly scaling global logistics platform. You are tasked with migrating the company's monolithic relational backend to a distributed No-SQL architecture utilizing MongoDB to handle massive data volumes.*

## Phase 2: Schema Design & The "Change of Chip"

### Legacy: Normalized Relational

Redesign the fractured schema consisting of:

- Customer & Driver Profiles
- Route & Delivery\_Stop chains
- Granular Package\_Log tables

### Modern: No-SQL Hierarchy

- Construct new document models; defend choices on **Embedding vs. Referencing**.
- Optimize for the **physical sizing constraints** of the database engine (16MB BSON limit).
- Account for read/write patterns in high-velocity logistics.

# Closing Activity: Monolith to Distributed NoSQL

*General Scenario Setup: Welcome to your new role as Lead Data Architect for a rapidly scaling global logistics platform. You are tasked with migrating the company's monolithic relational backend to a distributed No-SQL architecture utilizing MongoDB to handle massive data volumes.*

## Phase 3: BSON Implementation & Logical Organization

### Physical BSON Implementation

Translate proposed logical JSON models into physical BSON by:

- Detailing and justifying exact **extended data types** (e.g., Decimal128 for fees, Date for TTL).
- Mapping critical fields to optimized binary representations to minimize storage footprint.

### Logical Collection Strategy

Present a technical defense for organization across:

- **Dynamic Telemetry:** Strategy for massive arrays.
- **Static Profiles:** Optimization for infrequent writes.
- **Unstructured Logs:** Handling variability without compromising index efficiency.